

Capítulo 6: Class Design (Diseño de Clases)

1. **E.** Los constructores solo pueden llamar a `this()` o `super()` en la primera línea. Las otras opciones intentan llamarlos como métodos normales o después de otras sentencias.
2. **A, B, F.** El modificador `final` es compatible con `private` y `static`. Sin embargo, `abstract` es incompatible con ellos porque un método abstracto debe ser sobrescrito para ser útil.
3. **B, C.** Métodos sobrescritos (`overridden`) y ocultos (`hidden`) deben tener la misma firma (nombre y parámetros). El tipo de retorno puede ser covariante (un subtipo).
4. **F.** No compila porque el padre `Mammal` no define un constructor sin argumentos. El hijo `Platypus` está obligado a llamar a `super(int)` en su primera línea.
5. **E.** Las variables de instancia no se sobrescriben, se ocultan. El valor que se accede depende del tipo de referencia (`Speedster`), no del objeto real en memoria.
6. **D, E.** Para ser inmutable, la clase debe ser `final`. `Moose` falla porque no inicializa su campo `final`, y las demás permiten subclases mutables por no ser `final`.
7. **A.** Por polimorfismo, Java ejecuta el método del objeto real (`Spider`). La llamada con `long` no coincide con la sobrescritura y usa la versión del padre.
8. **D.** Los métodos `private` no se heredan, así que no se sobrescriben. El orden es: Constructor Padre (`Wow-`) → Constructor Hijo (`Oh-`) → Método del objeto.
9. **B, E.** Un método sobrescrito puede declarar nuevas excepciones no verificadas (`unchecked`). Si el original retorna `void`, el nuevo debe retornar `void` por fuerza.
10. **A, C.** Sin constructor por defecto en el padre, el hijo debe invocar uno existente con `this(...)` o `super(...)`. Las opciones A y C usan sintaxis válida de encadenamiento.
11. **C.** Orden de inicialización: 1. Bloques de instancia (`tac`) → 2. Constructor llamado por `this()` (`b`) → 3. Cuerpo del constructor original (`f`). Resultado: `tacbf`.
12. **C.** Tiene 3 errores: falta `super()`, un método `static` no puede ser sobrescrito por uno de instancia, y el retorno `Number` no es covariante de `Integer`.
13. **A, G.** El constructor por defecto solo aparece si no defines ningún constructor. En G, `bird()` es un método porque tiene tipo de retorno.

14. **B, E, F.** Las clases pueden implementar múltiples interfaces. Si la clase C implementa la interfaz D, C es técnicamente un subtipo de D.
15. **C.** El método `isBlind()` en la clase abstracta falla porque no tiene cuerpo ni está marcado con el modificador `abstract`.
16. **D.** Los componentes `static` corren una sola vez (Padre → Hijo). Luego, los de instancia corren con cada `new`. Resultado: `uq uq uqcrqm`.
17. **C, F.** `this` es accesible en cualquier método de instancia. `main()` puede entrar a constructores `private` si está dentro de la misma clase.
18. **D, F.** `drink()` es un método oculto (ambos `static`). `dance()` es una sobrecarga porque cambia el tipo de parámetro (`String` vs `CharSequence`).
19. **F.** Falla porque `Lizard` no invoca explícitamente al constructor del padre que recibe un `int`, y el padre no tiene uno vacío.
20. **E.** `fly()` se sobrescribe con retorno covariante. El polimorfismo ejecuta la versión de `Macaw`, que cambia el valor de `feathers` a 3.
21. **B, G.** Los objetos inmutables no deben permitir extensiones (clase `final`). Se pueden leer datos mutables si se devuelve una copia defensiva.
22. **D.** Las variables se ocultan. El valor depende de la referencia: `m.name` usa la de `Child` y `t.name` usa la de `Person`.
23. **B.** El flujo de constructores va de "arriba hacia abajo": `Canine` → `Fox` → `Fennec`. El resultado de los `append` es `qpzj`.
24. **C.** Sigue el orden estricto: `Static (Padre→Hijo)` → `Instancia/Constructor (Padre)` → `Instancia/Constructor (Hijo)`. Resultado: `182493`.
25. **B, C.** Una clase concreta debe implementar todo lo abstracto heredado y puede marcarse como `final` para cerrar la jerarquía.
26. **D.** Al convertir el objeto a la referencia del padre (`Whale`), ya no puedes llamar a métodos que solo existen en el hijo (`dive(int...)`).

Capítulo 7: Beyond Classes (Interfaces, Enums, Records)

1. **B, D.** Los Records son implícitamente final. Chameleon falla por ser abstract e Iguana por declarar campos estáticos que chocan con los del header.
2. **A, B, D, E.** La referencia puede ser de cualquier supertipo (clase padre o interfaz). var es legal porque el compilador detecta el tipo.
3. **C.** En un Enum, si incluyes campos o métodos, la lista de constantes debe terminar con punto y coma (;).
4. **C.** Al extender una clase sealed, el hijo debe ser obligatoriamente final, sealed o non-sealed. Armadillo no tiene ninguno.
5. **E.** Beetle no implementa getNumberOfSections() porque cambió la firma (añadió un parámetro), lo cual es sobrecarga, no implementación.
6. **D, E.** Métodos abstract no pueden tener cuerpo. Una clase implementa (implements) interfaces, no las extiende.
7. **E.** Los métodos de interfaz son public. Al sobrescribirlos, no se puede reducir su visibilidad (por ejemplo, a package-private).
8. **A, B, C.** Un buen encapsulamiento requiere campos private. Los getters y setters pueden ser public o protected según el diseño.
9. **A, E, F.** Se puede pasar cualquier objeto que sea un subtipo del esperado (Snake). null siempre es válido para tipos de referencia.
10. **A, B, C, E.** Los métodos private static de una interfaz no se heredan, por lo que no causan conflictos de firma en las clases hijas.
11. **B.** Java 16 permite miembros estáticos en clases internas. El código accede correctamente a la variable de la clase anidada.
12. **A, B, E.** El encapsulamiento protege los datos mediante campos private y permite acceso controlado a través de métodos.
13. **F.** En un switch de Enum, las etiquetas case solo deben usar el nombre del valor (SPRING), sin el nombre de la clase.
14. **A, C, E.** Las sealed interfaces controlan qué otras interfaces las extienden. Las clases hijas pueden ser non-sealed para abrir la jerarquía.

15. **G.** No compila porque Spirit es una clase final. No se permiten subclases (ni siquiera anónimas) de tipos final.
16. **E.** Una clase anidada static no tiene acceso a miembros de instancia (count) de la clase externa sin una referencia al objeto.
17. **E, G.** Los métodos con cuerpo en interfaces deben ser default o private. Las interfaces no soportan el modificador protected.
18. **E.** main() es static y no puede instanciar la clase interna Diet porque esta requiere una instancia activa de la clase externa Deer.
19. **G.** Si el Enum declara un método abstract, todos los valores deben implementarlo. Aquí solo uno lo hace.
20. **A, D, F.** El polimorfismo decide el método en tiempo de ejecución. Marcar un método como final impide que sea sobrescrito u ocultado.
21. **C, E.** En un record, los campos son final. No puedes usar super.brand para asignar valores; las validaciones van en el constructor compacto.
22. **C, D, F, G.** Para crear una inner class desde un método estático necesitas un objeto (new Lion().new Cub()). Las static classes no lo necesitan.
23. **D.** Para invocar un método default de una interfaz desde una clase que la implementa, usa: Interfaz.super.metodo().
24. **B, E, F.** Los métodos estáticos de interfaz no pueden llamar a métodos de instancia (getName). sneak() no puede llamar a roar() por ser estático.
25. **B.** La clase local accede a Zebra.this.x (la variable de la clase externa), por lo que el resultado es 24.
26. **G.** El código es legal. Los enums pueden tener constructores private y métodos que los valores individuales sobrescriban.
27. **B, C, D, G.** El compilador genera automáticamente: accessors, toString(), equals() y hashCode(). El constructor generado tiene argumentos.
28. **F.** Todas las opciones fallan por: métodos abstractos sin marcar, modificadores prohibidos (private abstract) o clases no abstractas con métodos abstractos.

29. **G** . Hay errores de sintaxis graves: el uso de `this().age` es inválido y las llamadas a constructores no llevan esos paréntesis extra.
30. **C**. `EmperorTamarin` no compila porque, al extender una sealed class, debe declararse explícitamente como `final`, `sealed` o `non-sealed`